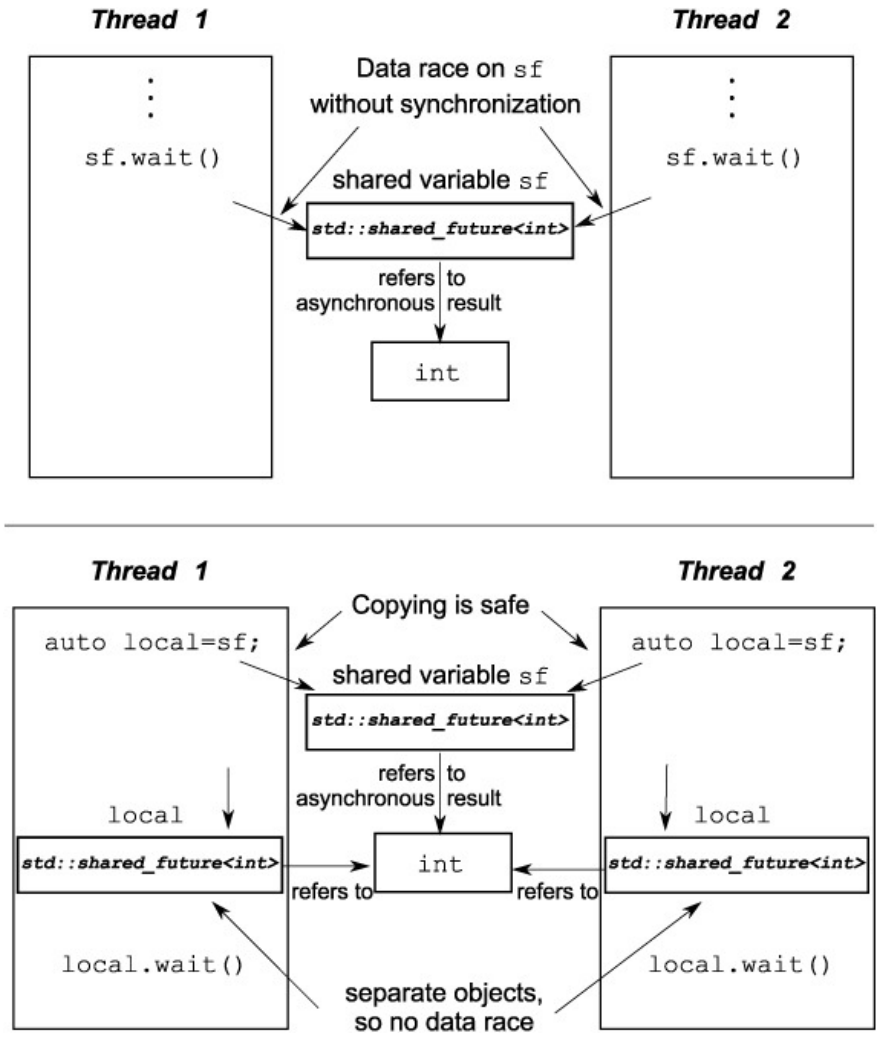
Now, with `std::shared_future`, member functions on an individual object are still unsynchronized, so to avoid data races when accessing a single object from multiple threads, you must protect accesses with a lock. The preferred way to use it would be to take a copy of the object instead and have each thread access its own copy. Accesses to the shared asynchronous state from multiple threads are safe if each thread accesses that state through its own `std::shared_future` object. See figure 4.1.

Figure 4.1. Using multiple `std::shared_future` objects to avoid data races



One potential use of `std::shared_future` is for implementing parallel execution of something akin to a complex spreadsheet; each cell has a single final value, which may be used by the formulas in multiple other cells. The formulas for calculating the results of the dependent cells can then use a `std::shared_future` to reference the first cell. If all the formulas for the individual cells are then executed in parallel, those tasks that can proceed to completion will do so, whereas those that depend on others will block until their dependencies are ready. This will thus allow the system to make maximum use of the available hardware concurrency.